

Shape Servoing of Deformable Objects

Hands-on Lab Session

Companion lab to the notes *An Introduction to Jacobian-Based Shape Servoing*

Estimated duration: 2.5–3 hours

Overview

In this lab we will develop a shape controller that brings a deformable object to a desired target shape, without assuming any mechanical model of the object. The object is a planar elastic rod held at both ends by two grippers, each with three degrees of freedom (DoF). The grippers impose end-effector (EE) poses on the rod; the rod settles into an equilibrium shape; a perception step samples a small set of points along it; and the controller adjusts the EE poses, step by step, until those sampled points reach a predefined target configuration.

The controller's actuation space is the six-dimensional end-effector pose vector $\mathbf{r} = [x_L, y_L, \theta_L, x_R, y_R, \theta_R]^\top$, and its feature space is the ten-dimensional sampled-point vector $\mathbf{s} = [x_1, y_1, \dots, x_5, y_5]^\top$. The method centres on the *interaction Jacobian* $\mathbf{J}$, the local linear map relating a gripper motion $\dot{\mathbf{r}}$ to the change $\dot{\mathbf{s}}$ it produces in the measured shape. Across the exercises we will estimate it from data, invert it to close a control loop, keep it accurate as the rod deforms, and analyse the three characteristic ways in which the loop fails. Together, the exercises build a complete (if minimal) data-driven shape-servoing pipeline that runs in two phases (Figures ?? and ??). In the *offline* phase, the robots run a short exploration sequence and a least-squares fit produces an initial Jacobian estimate $\hat{\mathbf{J}}^{(0)}$ (Exercise 3). In the *online* phase, the controller turns the shape error $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$ into an action $\Delta \mathbf{r}$; the robots apply it as the EE pose $\mathbf{r}$ holding the rod; the rod settles into a new true state $\mathbf{x}$; a perception step maps $\mathbf{x}$ to the measured features $\mathbf{s}$; and a Broyden update keeps $\hat{\mathbf{J}}$ current (Exercise 4).

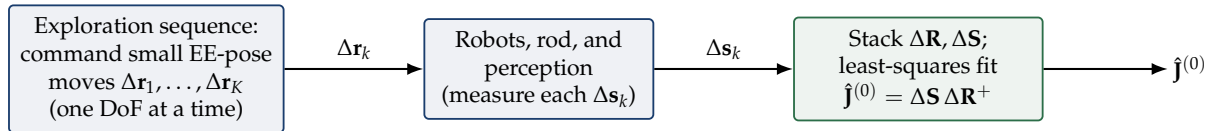


Figure 1: Offline initialisation. The robots run an exploration sequence, moving one EE degree of freedom at a time; each commanded move $\Delta \mathbf{r}_k$ and its measured feature response $\Delta \mathbf{s}_k$ are stacked into $\Delta \mathbf{R}, \Delta \mathbf{S}$ and fit by least squares to give the initial Jacobian estimate $\hat{\mathbf{J}}^{(0)}$.

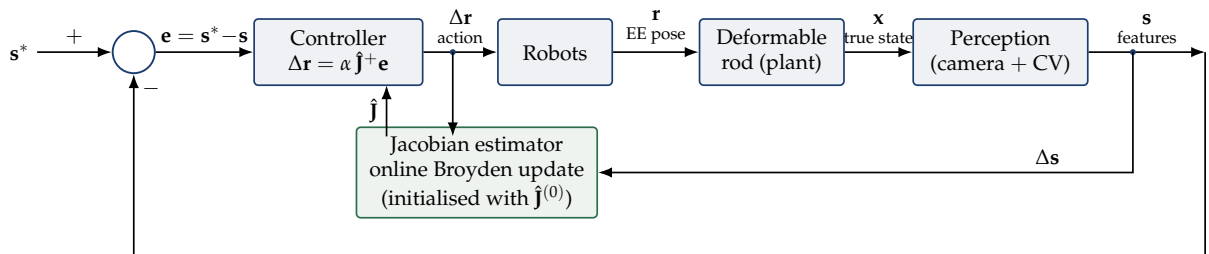


Figure 2: Online control loop. The forward path (blue) turns the shape error $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$ into an action $\Delta \mathbf{r}$, which the robots apply as the EE pose $\mathbf{r}$ holding the rod; the rod settles to a true state $\mathbf{x}$, which the perception block maps to the measured features $\mathbf{s}$, fed back (with a minus sign) to form the next error. The estimator (green) refines the Jacobian $\hat{\mathbf{J}}$ from $\Delta \mathbf{r}$ and $\Delta \mathbf{s}$, starting from $\hat{\mathbf{J}}^{(0)}$.

How the lab is organised. The lab is five short scripts, one per exercise, completed in order: `ex1_setup.py`, `ex2_representation.py`, `ex3_offline_jacobian.py`, `ex4_control_loop.py`, and `ex5_failure_modes.py`. Each one *imports* the functions you wrote in the earlier ones, so the controller is built up step by step and the code stays small. Fill in the lines marked `TODO`; the blocks marked “provided” (reporting and plotting) are not edited. The rod physics (`dlo.py`) and the plotting helpers (`labviz.py`) are provided too: the rod is a black box that returns the equilibrium shape for any commanded EE pose. Run a script with, e.g., `python ex3_offline_jacobian.py`; the figure each exercise should produce is reproduced alongside it below, so you can check your own result. Exercise 5 is a separate, diagnostic study and can be run on its own.

Checkpoint

Before you start (≈ 10 min). Create a Python environment and install the dependencies (`README.md` gives the per-operating-system commands):

```
python -m pip install -r requirements.txt
```

Then change into `code/` and check that the libraries and the rod back-end import without error:

```
cd code
python -c "import numpy, scipy, matplotlib, dlo, labviz"
```

You will work through `ex1_setup.py` to `ex5_failure_modes.py` in order. The figures appear when you run a script; if no display is available, each exercise still writes its figure to a `.png` file next to the script.

Notation (used throughout). Following the control convention, n is the feature-space (output) dimension and m the actuation (input) dimension. The rod’s true physical state is $\mathbf{x}$, its equilibrium configuration; a perception step maps $\mathbf{x}$ to the feature vector $\mathbf{s} \in \mathbb{R}^n$ ($n = 2P = 10$, from $P = 5$ sampled points stacked as $[x_1, y_1, \dots, x_P, y_P]$), which the controller uses. $\mathbf{r} \in \mathbb{R}^m$ ($m = 6$) is the end-effector (EE) pose we command (two 3-DoF gripper poses stacked). $\mathbf{s}^*$ is the target and $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$ the shape error. $\mathbf{J} \in \mathbb{R}^{n \times m}$ is the true interaction Jacobian ($\dot{\mathbf{s}} = \mathbf{J}\dot{\mathbf{r}}$), and $\hat{\mathbf{J}}$ is the estimate we actually use. $\mathbf{J}^+$ denotes the Moore–Penrose pseudo-inverse.

Objectives. After this lab you should be able to:

- explain what a shape feature vector is and how its choice changes which deformations the controller can measure;
- estimate an interaction Jacobian from motion data and characterise its rank and conditioning from the singular values;
- implement a pseudo-inverse shape controller and a Broyden online update;
- recognise the numerical signatures of the three failure modes of the loop.

1 The shape-servoing problem

≈ 15 min

Goal. Exercise 1 defines the controller’s actuation space and feature space, obtains the current and target feature vectors, and computes the shape error that the later feedback controller will reduce. We define a nominal EE pose $\mathbf{r}$ and a target pose $\mathbf{r}^*$; for each we solve the rod equilibrium and sample it to obtain the feature vectors $\mathbf{s}$ and $\mathbf{s}^*$; and we compute the shape error $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$ and its norm $\|\mathbf{e}\|$.

Tasks

In `ex1_setup.py`:

1. Choose a nominal grasp and a reachable target grasp (each gripper pose is $[x, y, \theta]$); pick a target that visibly moves or bends the rod, and try a few.
2. Complete `sampled_points` (solve the rod, then sample it), and form the feature vectors $\mathbf{s}, \mathbf{s}^*$ and the error $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$.

Checkpoint

The script reports $\dim \mathbf{r} = 6$, $\dim \mathbf{s} = 10$, and the initial error $\|\mathbf{e}\|$ (its value depends on the grasps you chose; the example in Figure ?? gives ≈ 1.3). Each arrow links a measured point to the target it must reach.

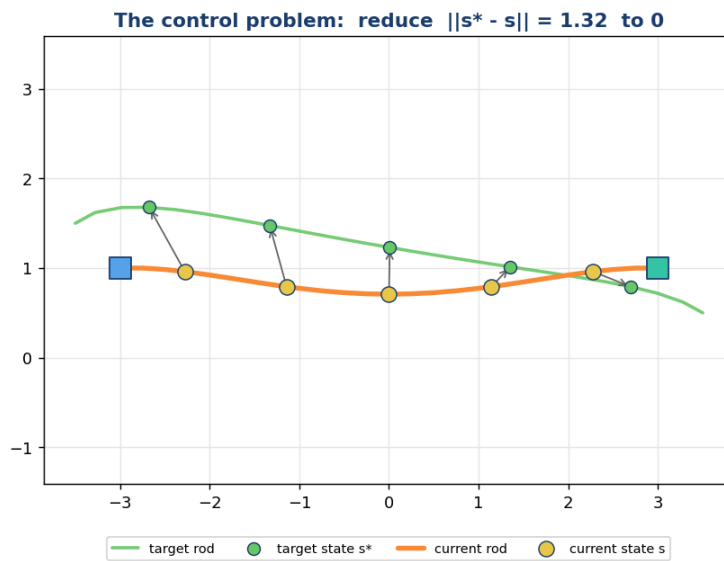


Figure 3: Expected output. The control problem: bring the measured state (yellow) onto the target state (green).

2 Choosing the shape representation

≈ 25 min

Goal. The feature vector $\mathbf{s}$ is a design choice, and different choices retain different information about the rod. We compare three representations of the same P sampled points $\mathbf{p}_1, \dots, \mathbf{p}_P$, and test how each responds to a *rigid transformation* (a translation and a rotation applied to the whole rod, without deforming it):

- **point coordinates** $\mathbf{s} = [x_i, y_i]$: absolute positions;
- **edge vectors** $\mathbf{e}_i = \mathbf{p}_{i+1} - \mathbf{p}_i$: invariant to translation;
- **curvature** $\kappa_i = \theta_{i+1} - \theta_i$ (turning angle between consecutive edges): invariant to any rigid motion.

Tasks

In `ex2_representation.py`:

1. Implement `rep_points`, `rep_edges`, `rep_curvature`, and `rigid_motion` (rotate the points about their centroid, then translate).

- Set FEATURE to the representation the controller will use in Exercises 3 and 4. The comparison and plot are provided.

Checkpoint

After a translation, $\|\Delta \mathbf{s}\|$ should be large for point coordinates and (numerically) zero for both edge vectors and curvature. After a translation *and* a rotation, only curvature should stay at zero (Figure ??). Explain *why* edge vectors are unchanged by a translation but change under a rotation.

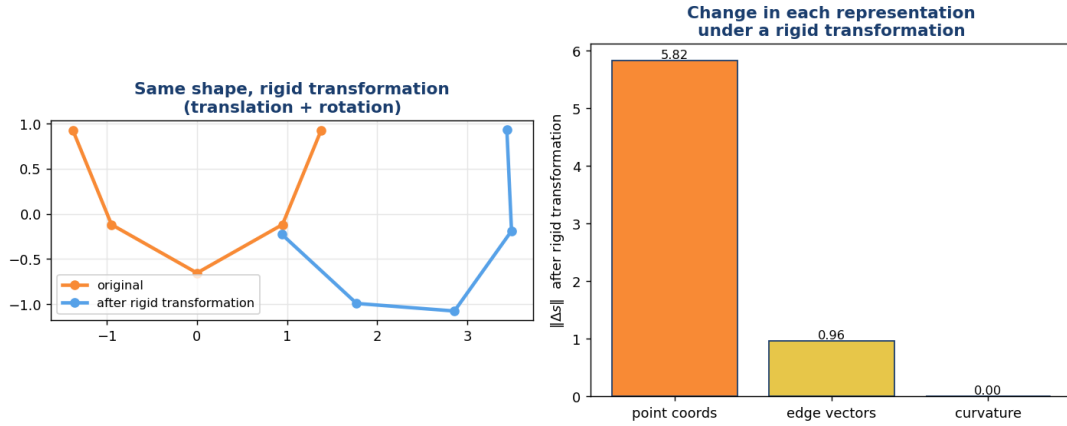


Figure 4: Expected output. The same shape under a rigid transformation; only the representations invariant to a given motion report $\|\Delta \mathbf{s}\| \approx 0$.

The three values have different units (metres vs. radians), so do not compare their magnitudes across representations. Read each only as “zero” or “non-zero”.

3 Offline Jacobian estimation

≈ 40 min

Goal. The controller never has a model of the rod, so it never computes a “true” Jacobian. Instead it *estimates* the interaction Jacobian from data: we move each gripper DoF a little, gather the input changes $\Delta \mathbf{R}$ ($m \times K$) and the output changes $\Delta \mathbf{S}$ ($n \times K$), and fit the single $\hat{\mathbf{J}}^{(0)}$ that best maps them by least squares,

$$\hat{\mathbf{J}}^{(0)} = \Delta \mathbf{S} \Delta \mathbf{R}^+.$$

We then read its rank and conditioning from the singular values $\sigma_1 \geq \dots \geq \sigma_6$: the *rank* is how many independent shape directions the grippers can produce; the *manipulability* $w = \prod_i \sigma_i$ is the overall actuation capacity; and the *condition number* $\kappa = \sigma_{\max} / \sigma_{\min}$ is the ratio between the most and least responsive directions.

Tasks

In `ex3_offline_jacobian.py`:

- Choose the exploration moves (`EXPLORE_AMP`, `EXPLORE_FRACS`).
- Complete `offline_jacobian`: gather $\Delta \mathbf{R}, \Delta \mathbf{S}$ from the exploration moves, then regress $\hat{\mathbf{J}}^{(0)} = \Delta \mathbf{S} \Delta \mathbf{R}^+$.
- In `svd_metrics`, compute the rank, manipulability w , and condition number κ from

the singular values.

Checkpoint

With point-coordinate features and the suggested moves, the offline fit residual is $\|\Delta \mathbf{S} - \hat{\mathbf{J}}^{(0)} \Delta \mathbf{R}\| \approx 0.31$, and the estimate has full column rank (rank = 6), manipulability $w \approx 0.20$, and condition number $\kappa \approx 17$. The figure shows $\hat{\mathbf{J}}^{(0)}$ as a heat-map and its six singular values (Figure ??).

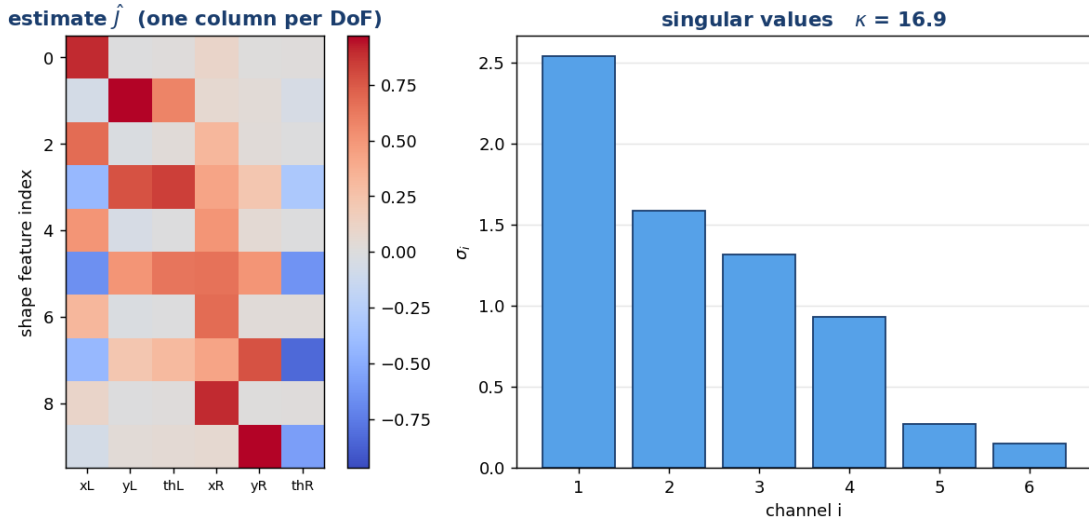


Figure 5: Expected output. The offline Jacobian estimate $\hat{\mathbf{J}}^{(0)}$ as a heat-map (one column per gripper DoF) and its six singular values.

4 Closing the loop with an online update

≈ 40 min

Goal. We put the estimate to work. Starting from the offline $\hat{\mathbf{J}}^{(0)}$ of Exercise 3, the controller closes the loop with the pseudo-inverse step

$$\Delta \mathbf{r} = \alpha \hat{\mathbf{J}}^+ (\mathbf{s}^* - \mathbf{s}), \quad \hat{\mathbf{J}}^+ = (\hat{\mathbf{J}}^\top \hat{\mathbf{J}})^{-1} \hat{\mathbf{J}}^\top,$$

and, because a fixed estimate drifts as the rod deforms, it refreshes $\hat{\mathbf{J}}$ after every step with the Broyden rank-one update

$$\hat{\mathbf{J}} \leftarrow \hat{\mathbf{J}} + \frac{(\Delta \mathbf{s} - \hat{\mathbf{J}} \Delta \mathbf{r}) \Delta \mathbf{r}^\top}{\Delta \mathbf{r}^\top \Delta \mathbf{r}}.$$

The control loop (high level). From the nominal grasp, repeat until the error is small: (1) measure $\mathbf{s}$ and form $\mathbf{e} = \mathbf{s}^* - \mathbf{s}$, and stop if $\|\mathbf{e}\|$ is small; (2) take the step $\Delta \mathbf{r} = \alpha \hat{\mathbf{J}}^+ \mathbf{e}$ and apply it ($\mathbf{r} \leftarrow \mathbf{r} + \Delta \mathbf{r}$); (3) measure the new shape and the change $\Delta \mathbf{s}$; (4) update $\hat{\mathbf{J}}$ with the Broyden rule, and record $\|\mathbf{e}\|$. The small gain $\alpha = 0.05$ makes the decay smooth and easy to read; try $\alpha = 0.3$ to compare.

Tasks

In `ex4_control_loop.py`:

1. Implement the Broyden update in `broyden_update`.
2. Complete the control loop in `main()`, following the pseudocode above. It starts from the offline estimate $\hat{\mathbf{J}}^{(0)}$ imported from Exercise 3.

Checkpoint

With the Broyden update on, the loop drives the error from ≈ 1.3 down to the tolerance in about 100 iterations (Figure ??). Now set `USE_BROYDEN = False` and re-run: the fixed offline estimate drifts as the rod deforms, so the same loop converges much more slowly. Updating $\hat{\mathbf{J}}$ online is what keeps convergence fast.

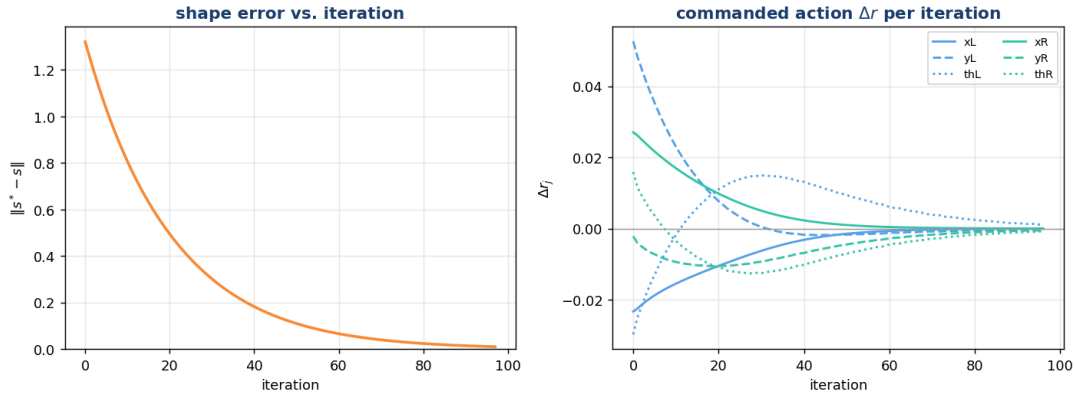


Figure 6: Expected output. The control loop driving the shape to the target: the error $\|\mathbf{s}^* - \mathbf{s}\|$ decays to zero (left) while the commanded action $\Delta \mathbf{r}$ shrinks as the error vanishes (right).

5 The three ways the loop fails

≈ 35 min

Goal. A shape-servoing loop relies on three conditions holding at once. Each one can fail, and each failure has a distinct numerical signature. In this exercise we reproduce all three. The conditioning analysis uses the rod's `true_jacobian` oracle, a Jacobian the controller never has, used here only to reveal the mechanics.

1. **Controllability (taut state).** Pull the rod straight by increasing the gripper separation. Some gripper motions stop producing any shape change, so the Jacobian loses rank from its *columns*: $\sigma_{\min} \rightarrow 0, w \rightarrow 0, \kappa \rightarrow \infty$.
2. **Observability (insensitive feature).** Move one gripper up and the other down by the same amount. The rod clearly deforms, but a symmetric scalar feature (the midpoint height) does not move: the feature map is insensitive to this deformation (a rank loss from the *rows*).
3. **Mechanical instability (buckling).** Compress the rod by reducing the gripper separation. At a critical point the smallest eigenvalue of the elastic-energy Hessian crosses zero; the straight equilibrium becomes unstable and the rod snaps. No finite linear $\mathbf{J}$ describes that jump.

Tasks

In `ex5_failure_modes.py` (it reuses your `svd_metrics` from Exercise 3):

1. In `taut_curve`, build the straight grasp at each separation, get its Jacobian from `rod.true_jacobian`, and read $\sigma_{\min}$, w and κ .
2. In `observability_curve`, build the antisymmetric motion (left gripper up by δ , right gripper down by δ).
3. In `buckling_curve`, query the rod for the smallest energy-Hessian eigenvalue at each separation (`rod.straight_min_eig`).

Checkpoint

The figures (Figure ??) should show κ rising from about 17 to 117 as the rod is pulled taut; the full state growing while the midpoint feature stays flat at zero; and the energy-Hessian eigenvalue crossing zero near a gripper separation of 6.0 (the rod's rest length), marking the onset of buckling.

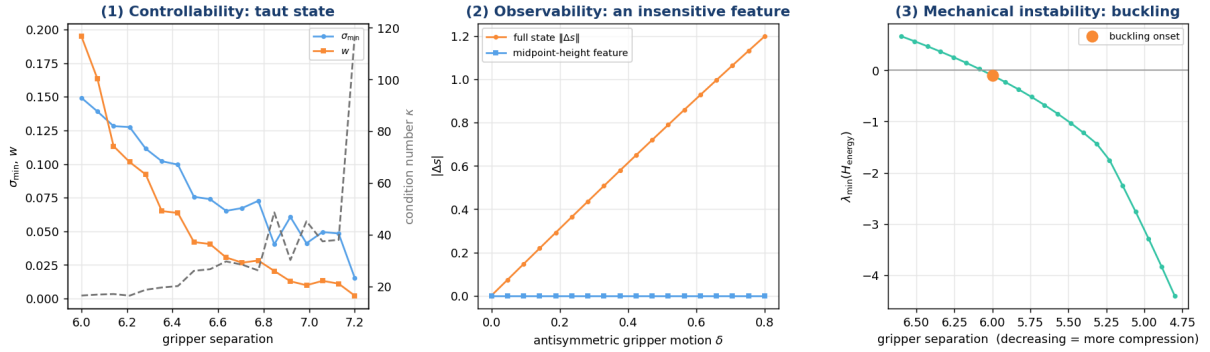


Figure 7: Expected output. Left to right: the taut-state collapse, the symmetric feature insensitive to the deformation, and the buckling eigenvalue crossing zero.

Wrapping up

Across the five exercises we have built every block of a model-free shape-servoing loop: a feature representation, a least-squares Jacobian estimate kept current online, a pseudo-inverse controller, and the diagnostics that reveal when the loop is about to fail. The same structure extends to richer features and real perception: what changes is how $\mathbf{s}$ is measured and how $\hat{\mathbf{J}}$ is estimated, not the structure of the loop itself.

Optional extensions. Any one of the following: (i) set FEATURE to curvature in Exercise 2 and see how the estimate and the convergence change; (ii) make the rod less stiff (Rod(k_bend=1.0)) and compare the convergence; (iii) in Exercise 4, plot $\|\hat{\mathbf{J}} - \mathbf{J}_{\text{true}}\|$ (using rod.true_jacobian) over the iterations to follow how the estimate tracks the true Jacobian.